

Gradientes, autodiferenciación y optimización

De la derivada local al ciclo reproducible de entrenamiento.

GRADIENTE

qué cambio local predice

BACKWARD

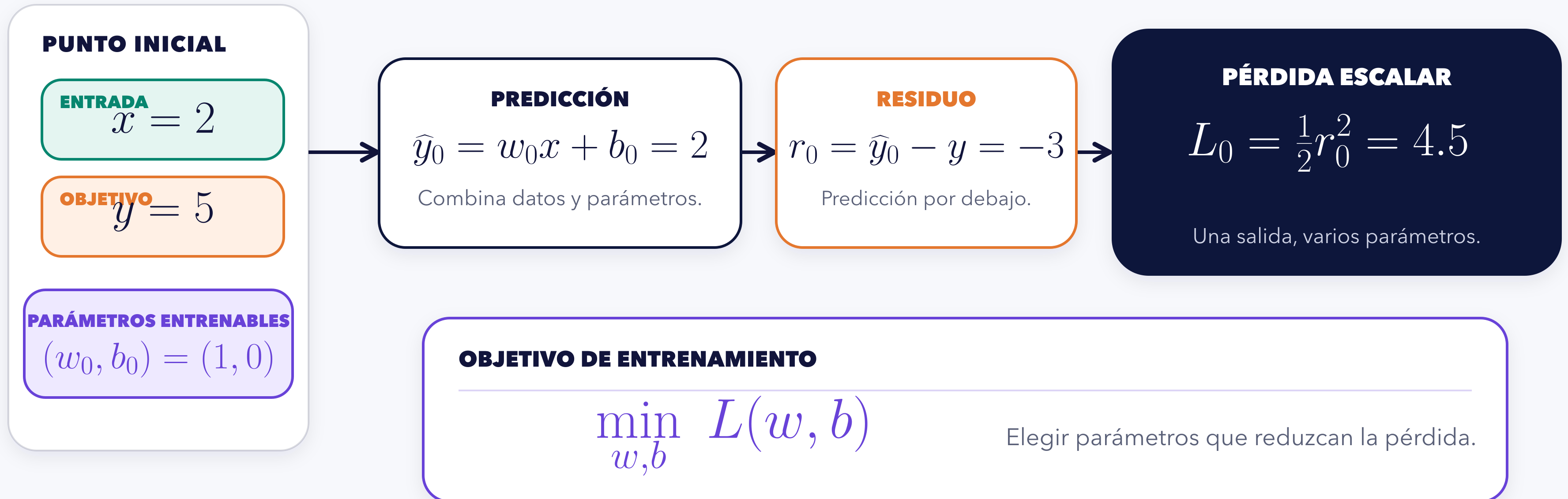
cómo se acumula la sensibilidad

OPTIMIZADOR

cómo usa el gradiente para actualizar

Una pérdida escalar organiza todo el entrenamiento

Datos y parámetros producen una sola cantidad que el entrenamiento intenta reducir.

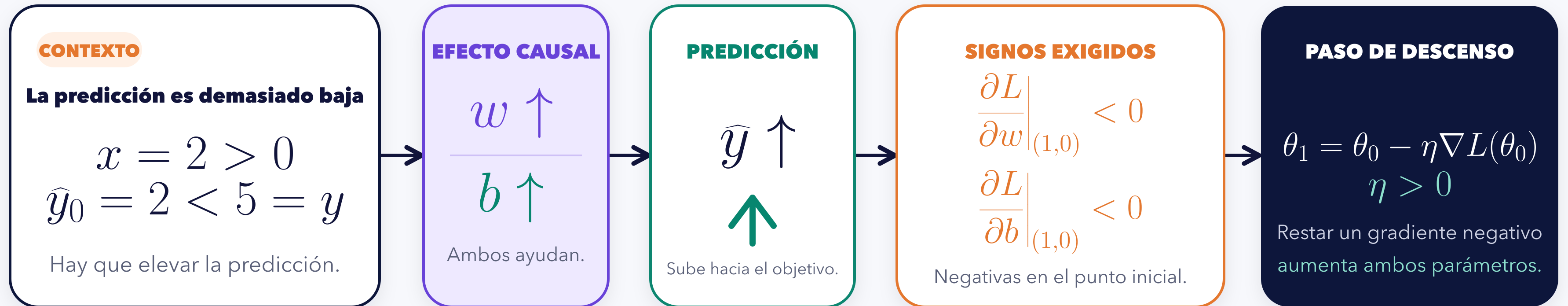


La salida escalar permite comparar cambios de muchos parámetros.

Una pérdida menor no demuestra por sí sola estabilidad, convergencia ni generalización.

El signo anticipa la derivada antes del cálculo

La predicción causal precede al álgebra: aquí el contexto $x > 0$ es indispensable.



CONTRASTE FALSABLE

Si la derivación produce signos positivos, contradice el efecto causal del caso conductor.

$$\Delta w > 0, \quad \Delta b > 0$$

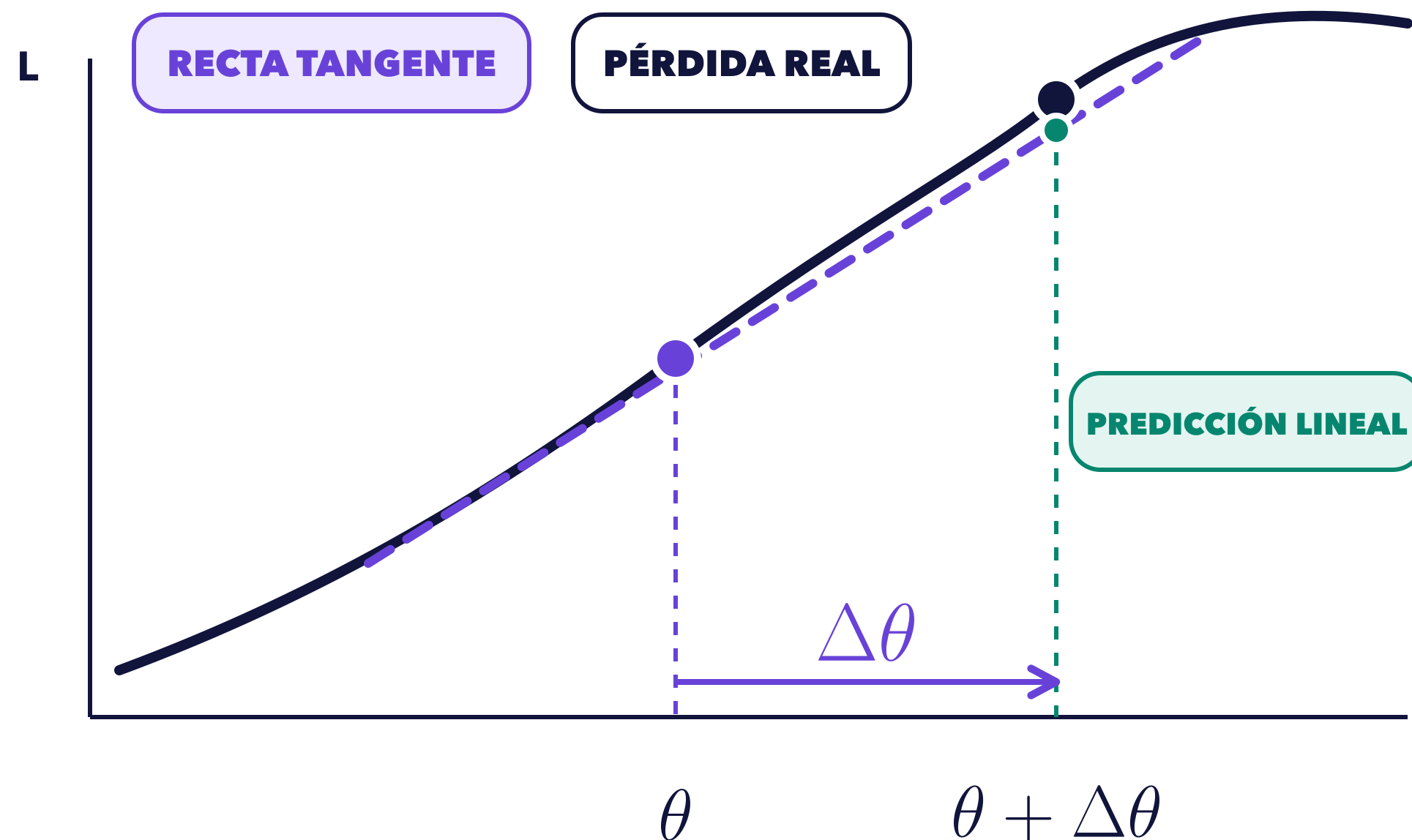
PREDICE ANTES DE DERIVAR

¿Qué cambiaría si x fuese negativo y la predicción siguiera siendo demasiado baja?

El gradiente predice cambios locales de la pérdida

La aproximación de primer orden sustituye una curva por su plano tangente cerca del punto actual.

UNA DIMENSIÓN: CURVA REAL Y RECTA TANGENTE



VARIAS DIMENSIONES: PLANO TANGENTE

$$L(\theta + \Delta\theta) \approx L(\theta) + \nabla L(\theta)^\top \Delta\theta$$

El gradiente reúne las pendientes parciales.

$$\nabla L(\theta) = \begin{bmatrix} \partial L / \partial \theta_1 \\ \vdots \\ \partial L / \partial \theta_p \end{bmatrix}$$

$$\Delta L_{\text{lin}} = \nabla L(\theta)^\top \Delta\theta$$

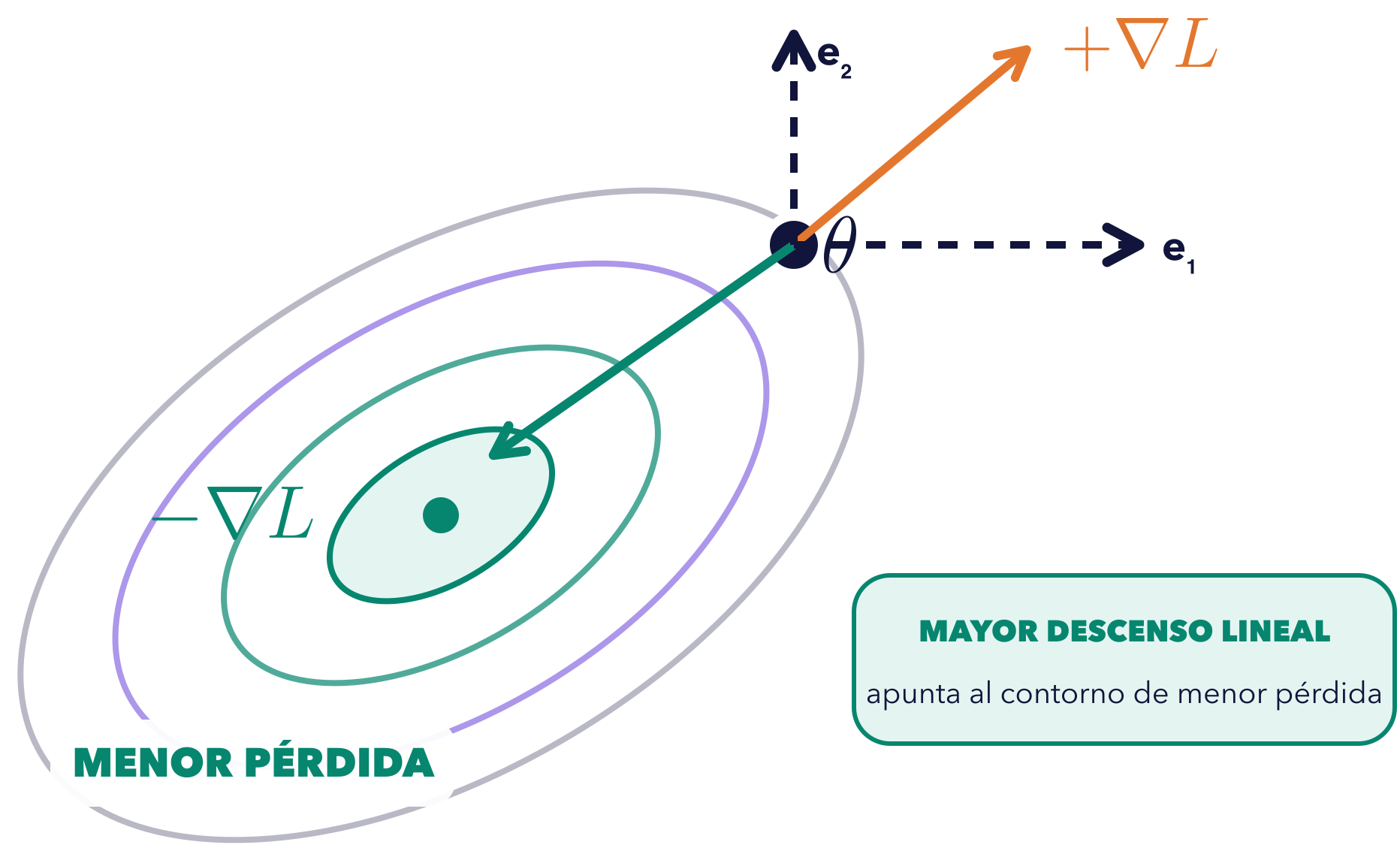
El producto interno proyecta el cambio elegido.

LÍMITE LOCAL La aproximación mejora cuando el cambio es pequeño; no garantiza el comportamiento global.

Descenso local máximo: contra el gradiente

Entre direcciones unitarias, Cauchy-Schwarz identifica la que reduce más la aproximación lineal.

PAISAJE LOCAL: MISMO PUNTO, CUATRO DIRECCIONES



RUTA MATEMÁTICA

$$D_d L(\theta) = \nabla L(\theta)^\top d$$

COMPARA EL SIGNO EN CUATRO DIRECCIONES

$d = e_1 : \partial_1 L$
signo depende

$d = +\nabla L$
 $\|\nabla L\|_2^2 > 0$

$d = e_2 : \partial_2 L$
signo depende

$d = -\nabla L$
 $-\|\nabla L\|_2^2 < 0$

COTA PARA PASOS UNITARIOS

$\|d\|_2 = 1$
 $\nabla L(\theta)^\top d \geq -\|\nabla L(\theta)\|_2$
 $\nabla L(\theta) \neq 0$

$d^* = -\frac{\nabla L(\theta)}{\|\nabla L(\theta)\|_2}$

IGUALDAD: DIRECCIÓN UNITARIA OPUESTA

ALCANCE

Si el gradiente es nulo, toda dirección unitaria tiene derivada cero y no existe una dirección única. La conclusión es local y euclídea; no promete el mejor paso finito ni el mínimo global.

La cuadrática contrasta dirección y cambio de pérdida

En el mismo punto, avanzar con el gradiente y avanzar contra él producen efectos opuestos.

CASO CONDUCTOR EN EL PUNTO INICIAL

$$L(w, b) = \frac{1}{2}(wx + b - y)^2$$

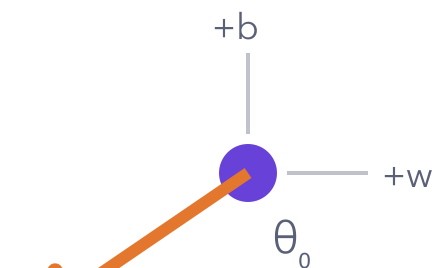
$$\theta_0 = (1, 0), \quad \nabla L(\theta_0) = (-6, -3)$$

ALINEARSE CON EL GRADIENTE

La predicción lineal es positiva.

$$\Delta\theta_+ = 0.05 \nabla L = (-0.30, -0.15)$$

$$\theta_+ = (0.70, -0.15)$$



$$\Delta L_{\text{lin}} = +2.25$$

LA PÉRDIDA AUMENTA

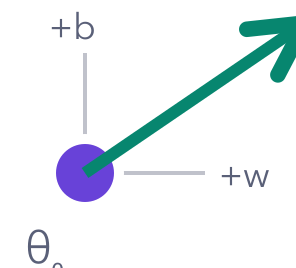
$$4.5 \rightarrow 7.03125$$

OPONERSE AL GRADIENTE

La predicción lineal es negativa.

$$\Delta\theta_- = -0.05 \nabla L = (0.30, 0.15)$$

$$\theta_- = (1.30, 0.15)$$



$$\Delta L_{\text{lin}} = -2.25$$

LA PÉRDIDA DISMINUYE

$$4.5 \rightarrow 2.53125$$

LECTURA LOCAL

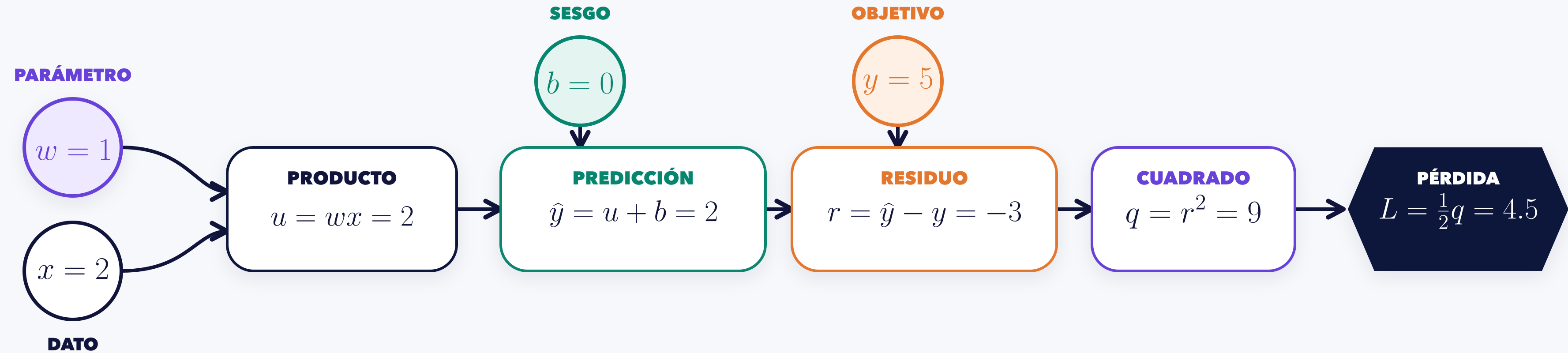
El signo del producto interno predice el cambio; el valor exacto confirma este paso pequeño.

La conclusión no autoriza extrapolar a pasos grandes ni a mínimos globales.

Backpropagation empieza por un forward descompuesto

Cada nodo conserva un valor y cada arista una derivada local que luego podrá componerse.

FORWARD: VALORES Y DEPENDENCIAS



RUTA HACIA w : PARCIALES LOCALES POR ARISTA

$$\frac{\partial u}{\partial w} = x = 2$$

$$\frac{\partial \hat{y}}{\partial u} = 1$$

$$\frac{\partial r}{\partial \hat{y}} = 1$$

$$\frac{\partial q}{\partial r} = 2r = -6$$

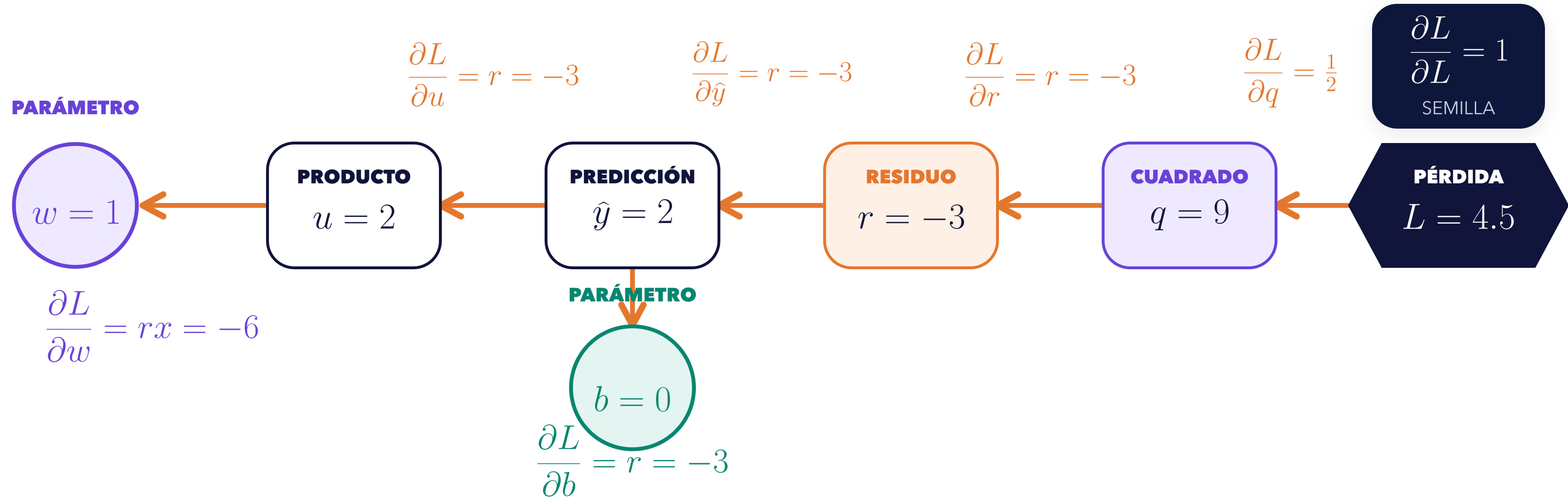
$$\frac{\partial L}{\partial q} = \frac{1}{2}$$

Las entradas x , b e y tienen sus propias parciales; aquí se sigue solo la ruta que termina en w .

La sensibilidad nace en 1 y viaja hacia atrás

Una semilla escalar se compone con derivadas locales hasta llegar a cada parámetro.

BACKWARD: PRODUCTOS LOCALES EN ORDEN INVERSO

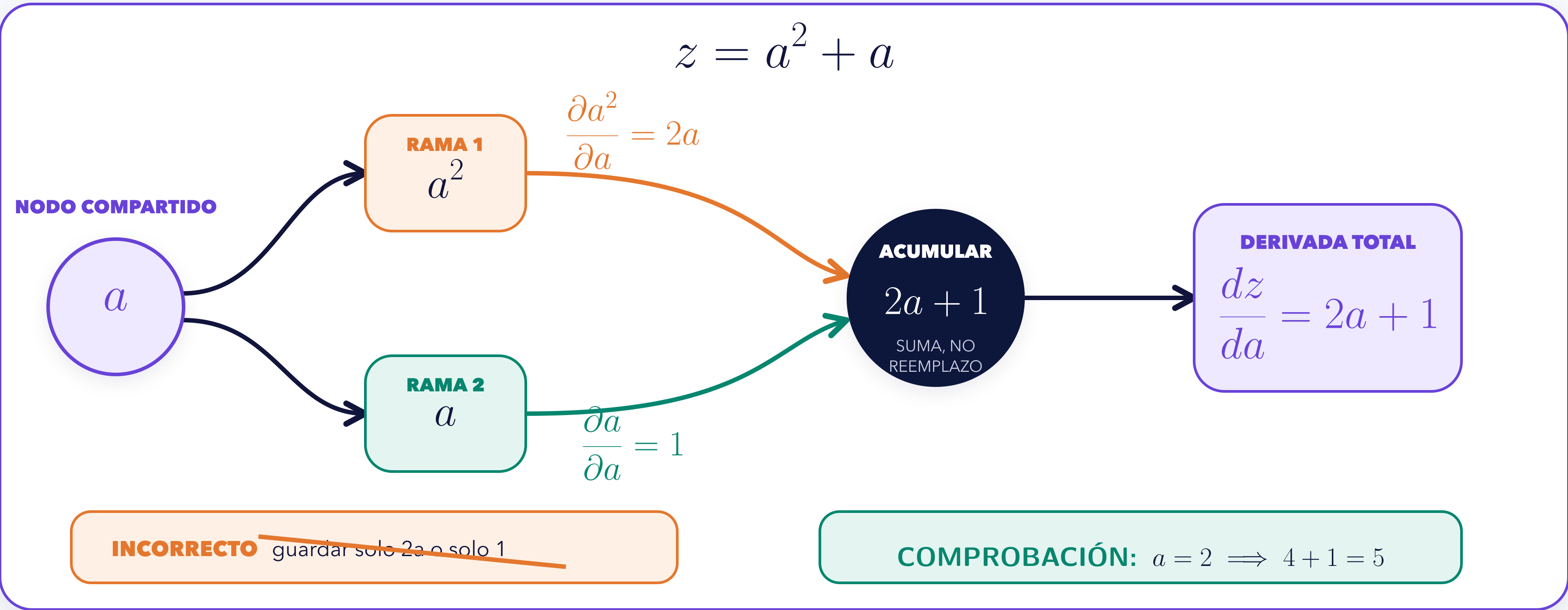


RECONSTRUYE ANTES DE MIRAR EL FINAL

Para llegar a w , multiplica $\frac{1}{2} \cdot 2r \cdot 1 \cdot 1 \cdot x$; para llegar a b , la última rama vale 1.

Dos rutas aportan; el gradiente se suma

Si a influye dos veces en z , cada rama entrega una contribución distinta a la derivada total.



La topología conserva las dos rutas; la suma conserva la derivada total.

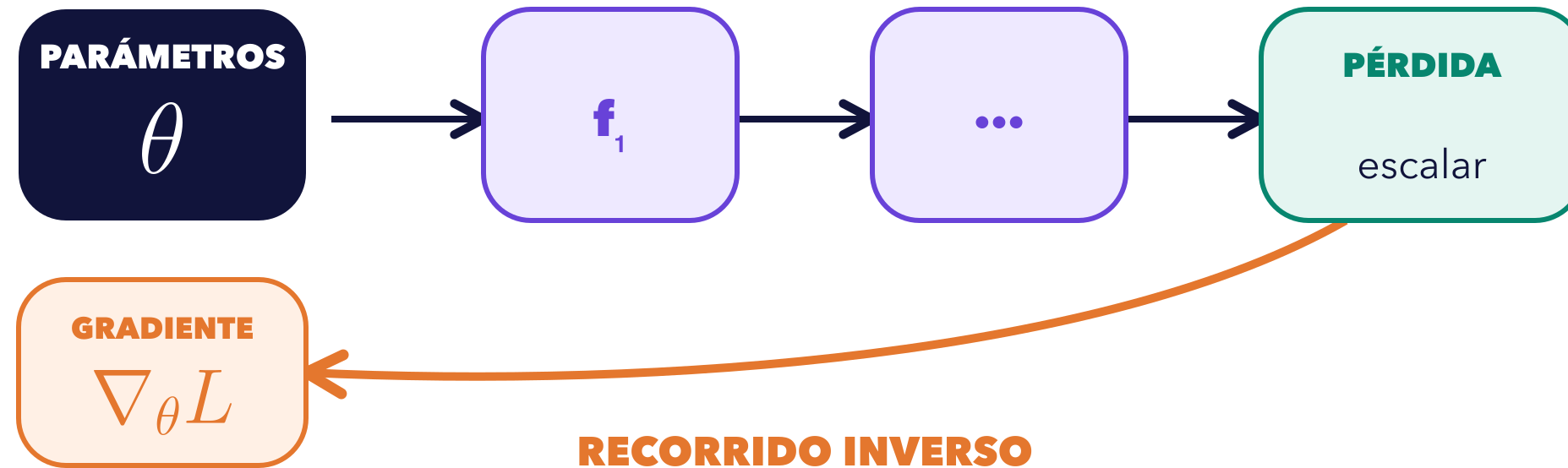
Backpropagation parte de una salida escalar

La regla de la cadena se organiza en modo inverso y reutiliza sensibilidades locales.

SALIDA ESCALAR: UNA SOLA SEMILLA

$$L = f_k \circ \dots \circ f_1(\theta) \in \mathbb{R}$$

Muchos parámetros alimentan una composición que termina en una cantidad.



SEMILLA ESCALAR: $\bar{L} = \frac{\partial L}{\partial L} = 1$

SALIDA NO ESCALAR

$$y(\theta) \in \mathbb{R}^m$$

La salida sola no fija qué combinación de sus componentes se diferencia.

SE DEBE ACLARAR: $v = \frac{\partial s}{\partial y}$



$v^T J_y(\theta)$
producto vector-Jacobiano

IDEA CENTRAL

Backpropagation no es otra regla: organiza la regla de la cadena sobre el DAG.

El modo inverso evita materializar el Jacobiano completo para obtener el gradiente de una pérdida escalar.

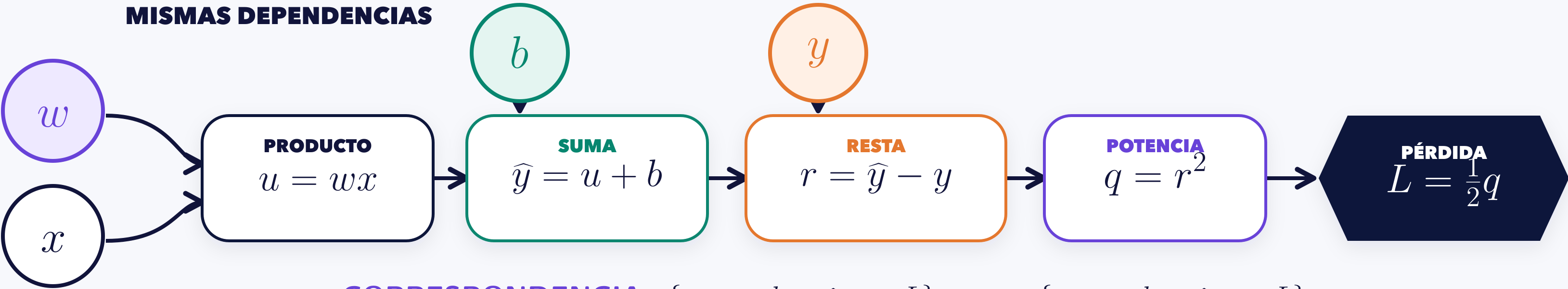
micrograd expone el mismo DAG escalar

No introduce otro problema: hace observables las piezas de la derivación manual.

RELACIÓN MANUAL

$$u = wx, \quad \hat{y} = u + b, \quad r = \hat{y} - y, \quad q = r^2, \quad L = \frac{1}{2}q$$

MISMAS DEPENDENCIAS



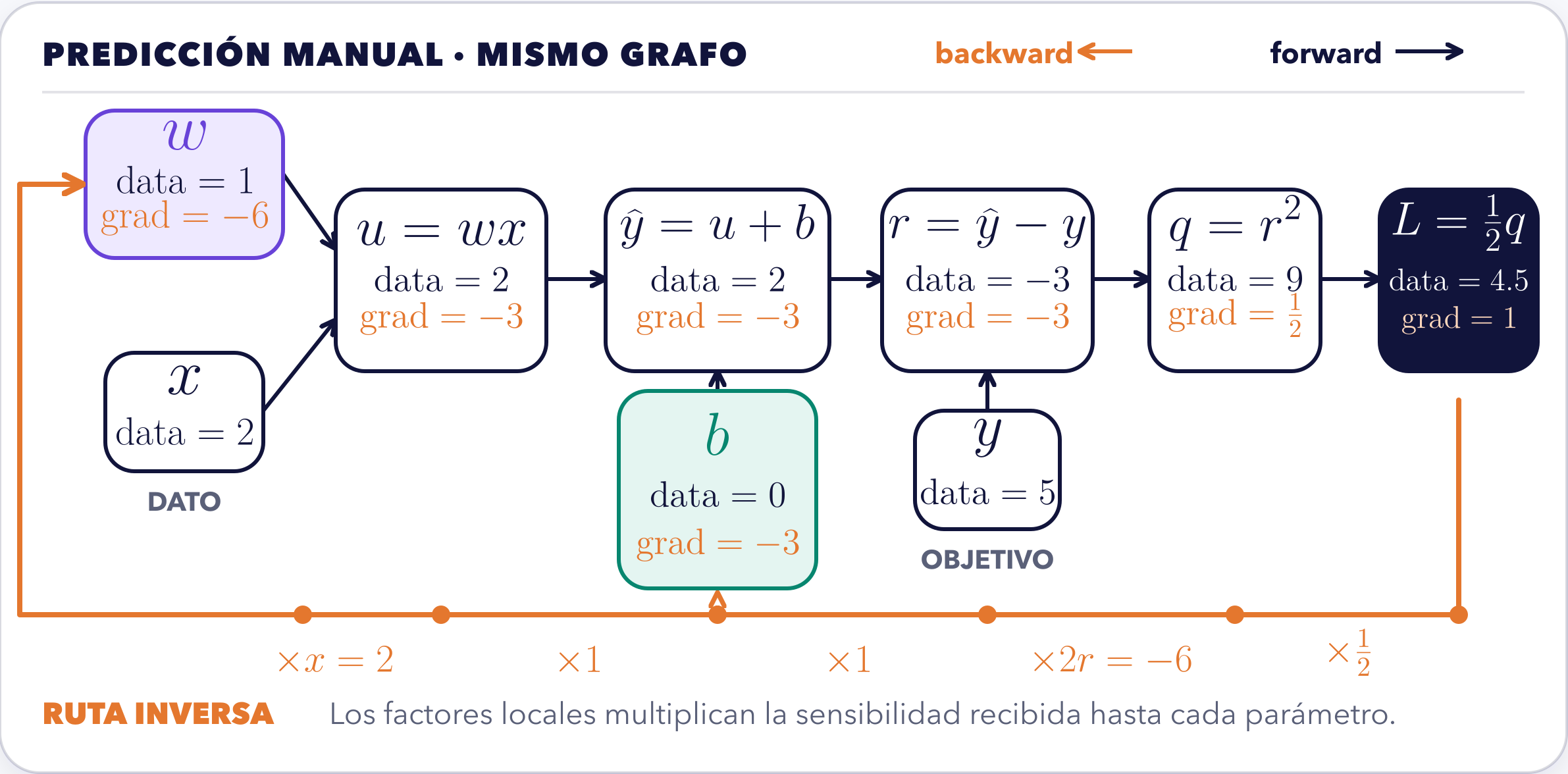
CORRESPONDENCIA: $\{x, y, w, b, u, \hat{y}, r, q, L\}_{\text{manual}} = \{x, y, w, b, u, \hat{y}, r, q, L\}_{\text{DAG}}$

CADA Value EXPONE CUATRO PIEZAS

VALOR	PADRES	OPERACIÓN LOCAL	GRADIENTE
resultado del forward	dependencias del DAG	regla que produjo el nodo	sensibilidad tras backward

La traza manual fija lo que micrograd debe reproducir

El código solo vale como evidencia si conserva el mismo DAG, los mismos valores y la misma propagación.

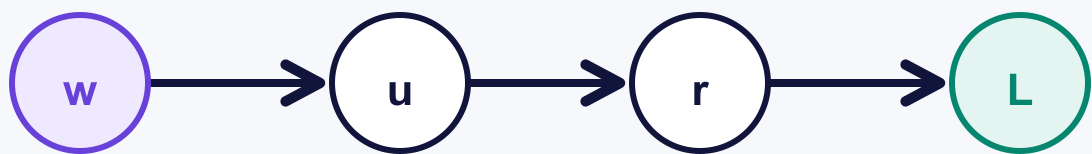


El orden topológico reúne contribuciones antes de propagar

Dependencias, sensibilidades y ramas requieren mecanismos distintos y coordinados.

1 • CONSTRUIR DEPENDENCIAS

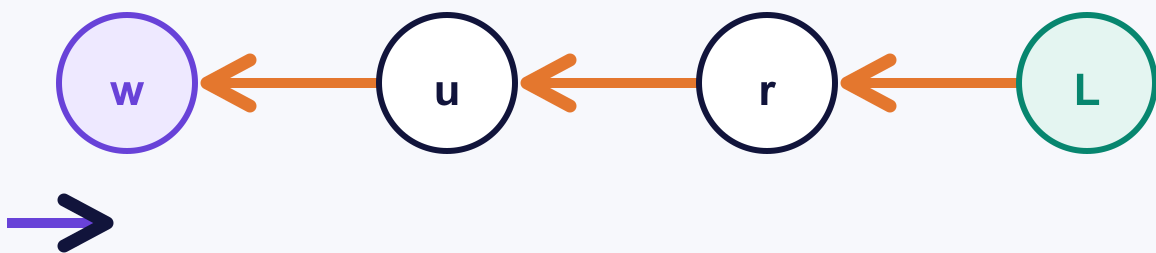
Un padre aparece antes que cada descendiente.



$$[w, x, u, b, \hat{y}, y, r, q, L]$$

2 • RECORRER AL REVÉS

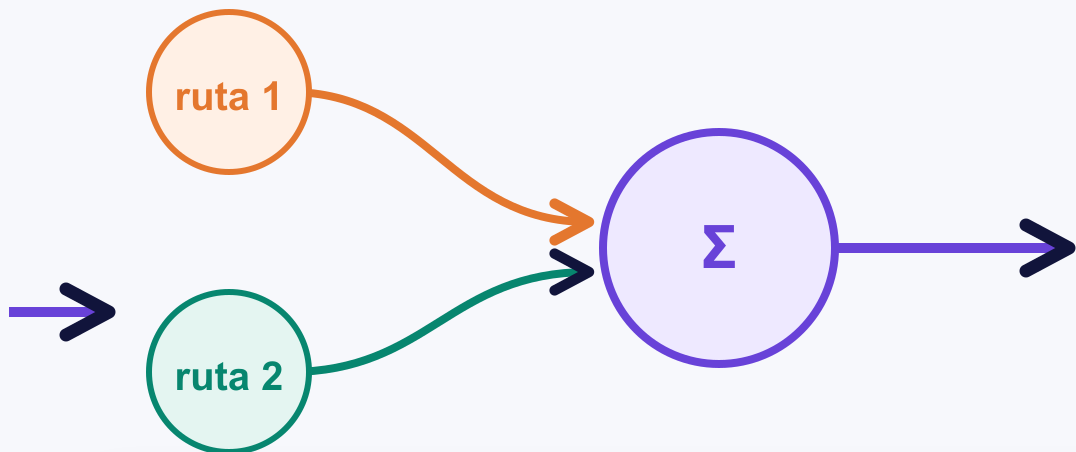
Los descendientes entregan primero su sensibilidad.



$$[L, q, r, y, \hat{y}, b, u, x, w]$$

3 • ACUMULAR RAMAS

Un nodo compartido suma antes de propagar.



$$\bar{a} = \bar{a}_{(a^2)} + \bar{a}_{(a)}$$

TRES RESPONSABILIDADES, UN SOLO BACKWARD

TOPOLOGÍA

resuelve dependencias

REGLA DE LA CADENA

resuelve sensibilidades

SUMA

resuelve ramas

El lote cambia las formas, no la pérdida escalar

Tres residuos atraviesan el mismo forward afín y se reducen a una sola autoridad.

CASO ESCALAR

$$\hat{y} = wx + b \longrightarrow L = \frac{1}{2}(\hat{y} - y)^2 \in \mathbb{R}$$

Una predicción produce un residuo y una pérdida.

CASO POR LOTES

El forward se vectoriza; la reducción final sigue siendo escalar.

DATOS Y PARÁMETROS

$$X = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}, \quad w_0 = 1, \quad b_0 = 0$$

FORWARD AFÍN

$$\hat{y} = Xw_0 + b_0\mathbf{1}_3 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

tres predicciones

RESIDUOS

$$r = \hat{y} - y = \begin{bmatrix} -2 \\ -3 \\ -4 \end{bmatrix}$$

tres errores

REDUCCIÓN

$$L_0 = \frac{1}{6}(4 + 9 + 16) = \frac{29}{6}$$

una pérdida

AUDITORÍA ANTES DE backward

FORMAS: $X : (3, 1) \quad y : (3,) \quad w : (1,) \quad b : () \quad \hat{y} : (3,) \quad L : ()$

TESIS CONSERVADA backward parte de una salida escalar explícita, aunque el forward opere por lotes.

Los gradientes se acumulan en las hojas del grafo

El forward crea conexiones; la pérdida escalar inicia el recorrido inverso.

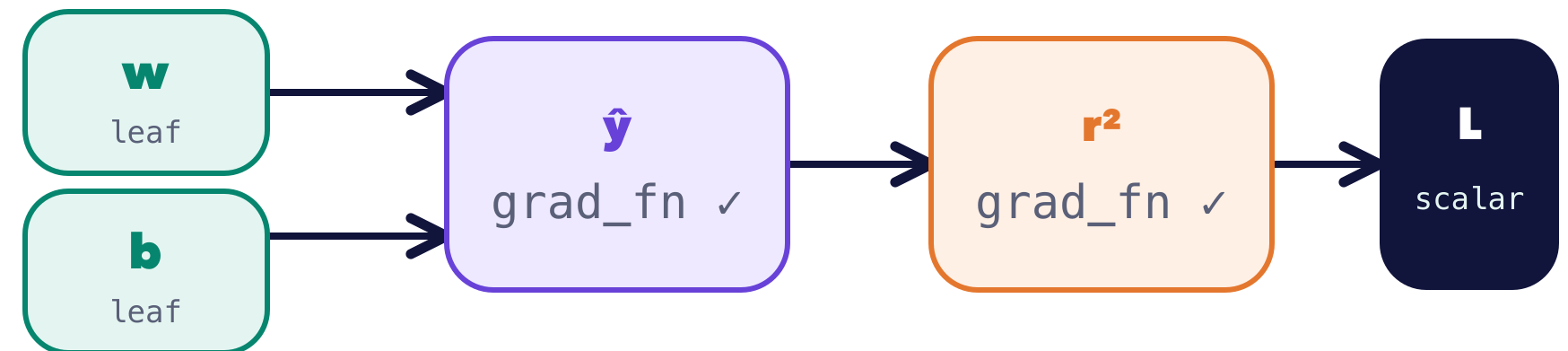
TRADUCCIÓN DIRECTA

```
X = torch.tensor([[1.], [2.], [3.]])
y = torch.tensor([3., 5., 7.])
w = torch.tensor([1.], requires_grad=True)
b = torch.tensor(0., requires_grad=True)
y_hat = X @ w + b
residual = y_hat - y
loss = 0.5 * torch.mean(residual ** 2)
loss.backward()
```

```
loss.shape == torch.Size([])
loss.grad_fn is not None
```

QUIÉN CONSERVA HISTORIAL

$$\hat{y} = Xw + b\mathbf{1}_3, \quad L = \frac{1}{2} \text{mean}((\hat{y} - y)^2) \in \mathbb{R}$$



DÓNDE QUEDA EL RESULTADO

$$w.\text{grad} = -\frac{20}{3}, \quad b.\text{grad} = -3$$

`w.grad_fn` is None

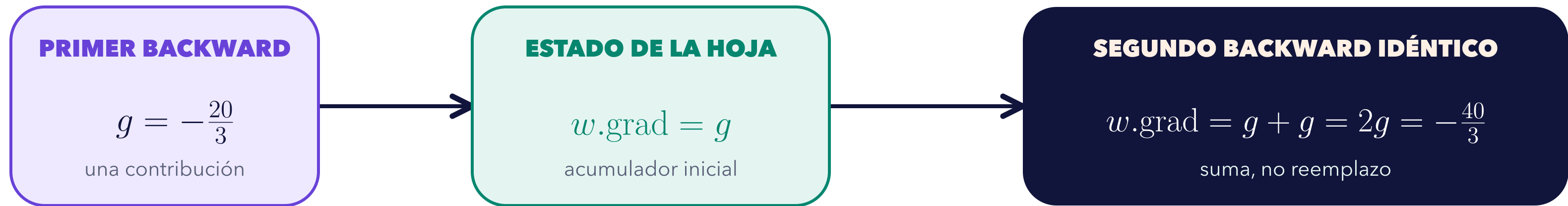
`b.grad_fn` is None

backward() recorre el grafo, pero acumula el resultado en las hojas.

Acumular y reiniciar son decisiones distintas

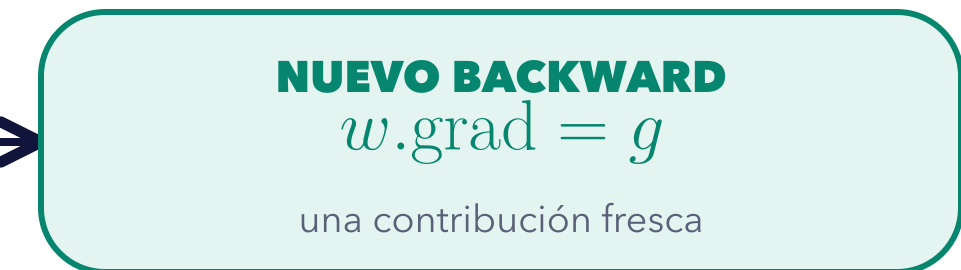
El recorrido inverso suma contribuciones; el reinicio fija el estado del siguiente recorrido.

SIN REINICIO: LA SEGUNDA CONTRIBUCIÓN SE SUMA



REINICIO EXPLÍCITO ANTES DEL NUEVO RECORRIDO

`optimizer.zero_grad(set_to_none=True):` $w.\text{grad} \leftarrow \text{None}$

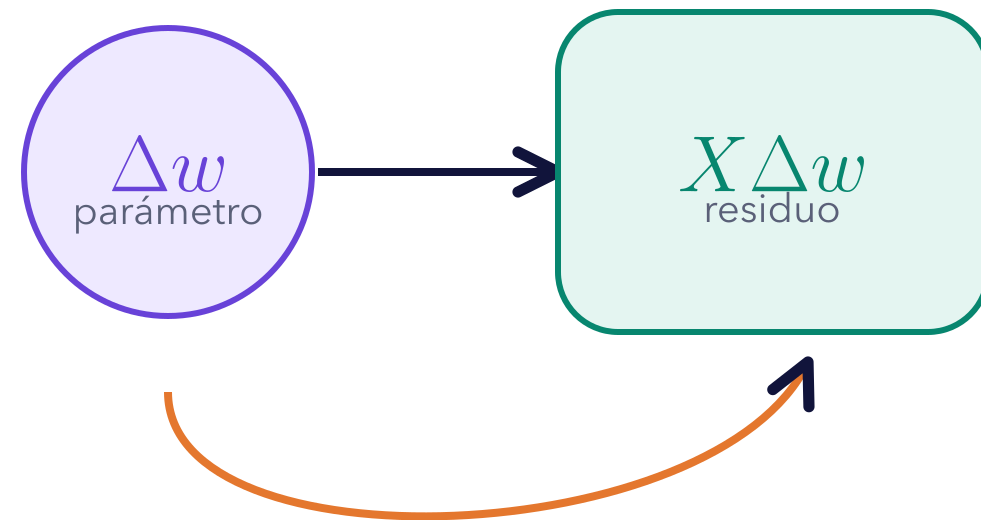


None $\neq$ tensor de ceros: conserva una señal sobre participación en el grafo.

El término lineal revela el gradiente por lotes

Perturbar el parámetro permite separar la dirección de cambio del resto cuadrático.

QUÉ CAMBIA AL MOVER EL PARÁMETRO



EL CAMBIO ENTRA A TRAVÉS DE X

El resto cuadrático se vuelve despreciable frente al término lineal cuando el cambio es pequeño.

EXPANDIR → CONSERVAR → REESCRIBIR

$$\begin{aligned} r(w + \Delta w, b) &= r + X\Delta w, \\ L(w + \Delta w, b) &= \frac{1}{2B}(r + X\Delta w)^\top (r + X\Delta w) \\ &= L(w, b) + \frac{1}{B}r^\top X\Delta w + \frac{1}{2B}\|X\Delta w\|_2^2, \\ \frac{1}{B}r^\top X\Delta w &= \left(\frac{1}{B}X^\top r\right)^\top \Delta w. \end{aligned}$$

LECTURA el vector que acompaña al cambio es el gradiente.
La transposición deja explícito el producto interno.

FORMAS: $X : (B, d)$, $r : (B)$, $X^\top r : (d)$, $\Delta w : (d)$

$$\nabla_w L = \frac{1}{B}X^\top r$$

MISMA FORMA QUE EL PARÁMETRO

Cada gradiente debe heredar la forma de su parámetro

La misma pérdida produce un vector para los pesos y un escalar para el sesgo.

PERTURBAR EL VECTOR DE PESOS

EL CAMBIO PASA POR LA MATRIZ DE DATOS

$$r(w + \Delta w, b) = r + X \Delta w,$$

$$\text{término lineal} = \frac{1}{B} r^\top X \Delta w,$$

$$\nabla_w L = \frac{1}{B} X^\top r = -\frac{20}{3} \in \mathbb{R}^d.$$

RESULTADO VECTORIAL

PERTURBAR EL SESGO ESCALAR

EL MISMO CAMBIO SE REPLICA EN TODO EL LOTE

$$r(w, b + \Delta b) = r + \Delta b \mathbf{1}_B,$$

$$\text{término lineal} = \frac{1}{B} r^\top \mathbf{1}_B \Delta b,$$

$$\frac{\partial L}{\partial b} = \frac{1}{B} \mathbf{1}_B^\top r = -3 \in \mathbb{R}.$$

RESULTADO ESCALAR

AUDITORÍA DE FORMAS: $X^\top r : (d \times B)(B) \rightarrow d = \text{shape}(w)$ $\mathbf{1}_B^\top r : (1 \times B)(B) \rightarrow 1 = \text{shape}(b)$

SI LAS FORMAS NO COINCIDEN, LA DERIVACIÓN O EL BROADCASTING EXIGEN REVISIÓN

Un paso explícito verifica la derivación antes de delegar

La resta actualiza parámetros; no vuelve a calcular el gradiente.

CÁLCULO ANTES DE EJECUTAR

$$\eta = 0.1, \quad (w_0, b_0) = (1, 0), \quad \nabla_w L = -\frac{20}{3}, \quad \frac{\partial L}{\partial b} = -3$$

$$w_1 = w_0 - \eta \nabla_w L = 1 - 0.1 \left(-\frac{20}{3} \right) = \frac{5}{3}$$

$$b_1 = b_0 - \eta \frac{\partial L}{\partial b} = 0 - 0.1(-3) = 0.3$$

$$\hat{y}_1 - \hat{y}_0 = X \left(\frac{2}{3} \right) + 0.3 \mathbf{1}_3 > 0$$

AMBAS PREDICCIONES SUBEN, COMO SE ANTICIPÓ

CONTRASTE COMPUTACIONAL MÍNIMO

```
with torch.no_grad():  
    w -= 0.1 * w.grad  
    b -= 0.1 * b.grad
```

RESULTADO

w = 1.6667

RESULTADO

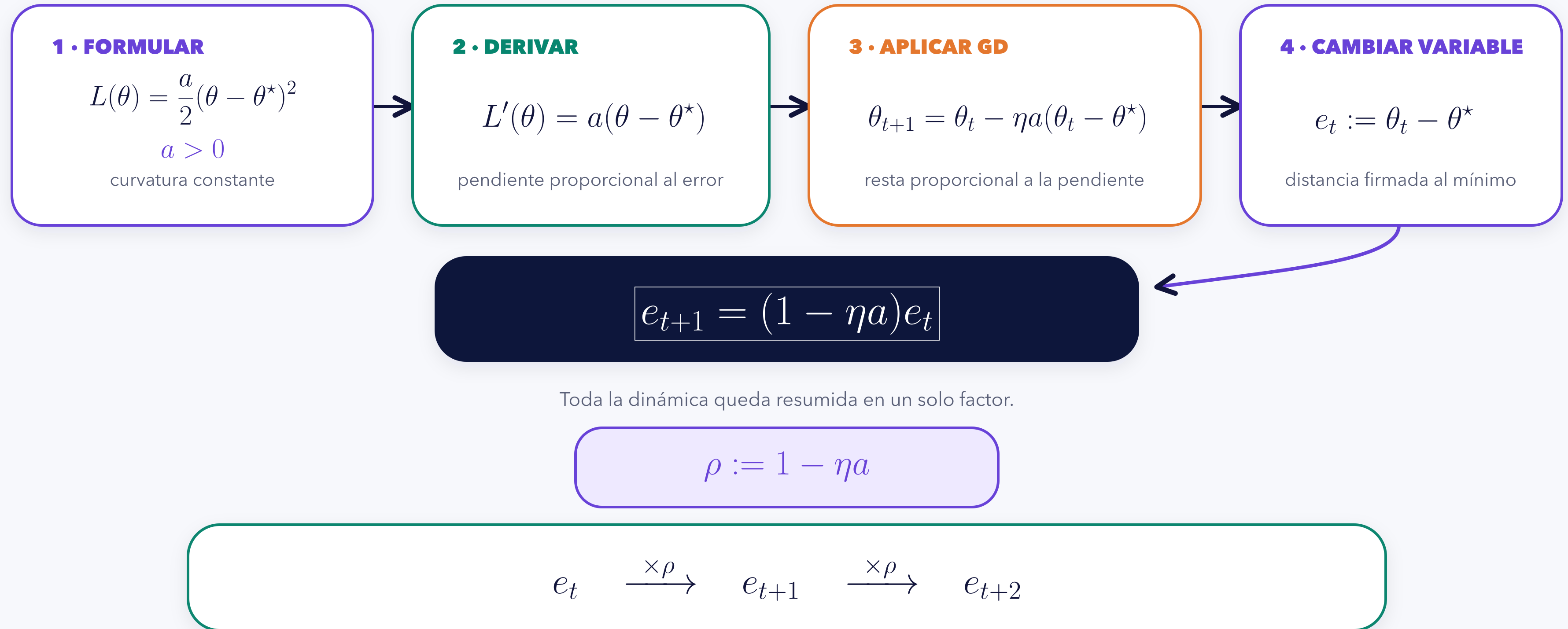
b = 0.3000

backward() produce gradientes.
no_grad() evita registrar la actualización.

EN UN CICLO REAL, optimizer.step() DELEGA ESTA MISMA RESPONSABILIDAD

La estabilidad de GD se decide en la recurrencia del error

La cuadrática convierte cada iteración en una multiplicación exacta.



CLASIFICAR GD SIGNIFICA ESTUDIAR EL SIGNO Y EL MÓDULO DE ESTE MULTIPLICADOR

El factor efectivo clasifica cinco regímenes de aprendizaje

Una tasa mayor puede acelerar, oscilar o destruir la convergencia.

$$\rho := 1 - \eta a$$

-1

0

1

$$\eta > \frac{2}{a} \iff \rho < -1$$

DIVERGE

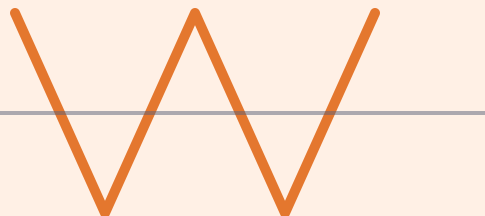
alterna y crece



$$\eta = \frac{2}{a} \iff \rho = -1$$

NO CONTRAE

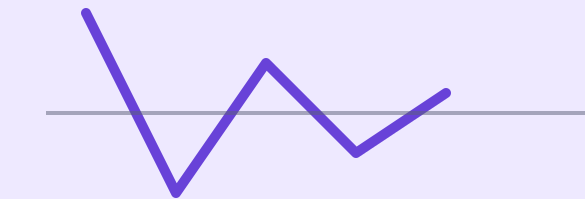
alterna con igual magnitud



$$\frac{1}{a} < \eta < \frac{2}{a} \iff -1 < \rho < 0$$

OSCILA Y CONVERGE

cambia de signo y decrece



trazas esquemáticas del error firmado

$$\eta = \frac{1}{a} \iff \rho = 0$$

UN PASO

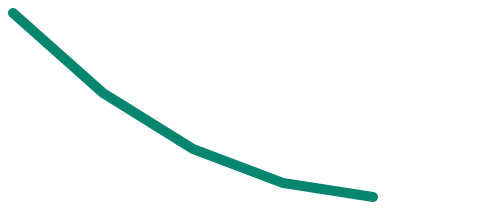
el error pasa a cero



$$0 < \eta < \frac{1}{a} \iff 0 < \rho < 1$$

MONÓTONA

conserva signo y decrece



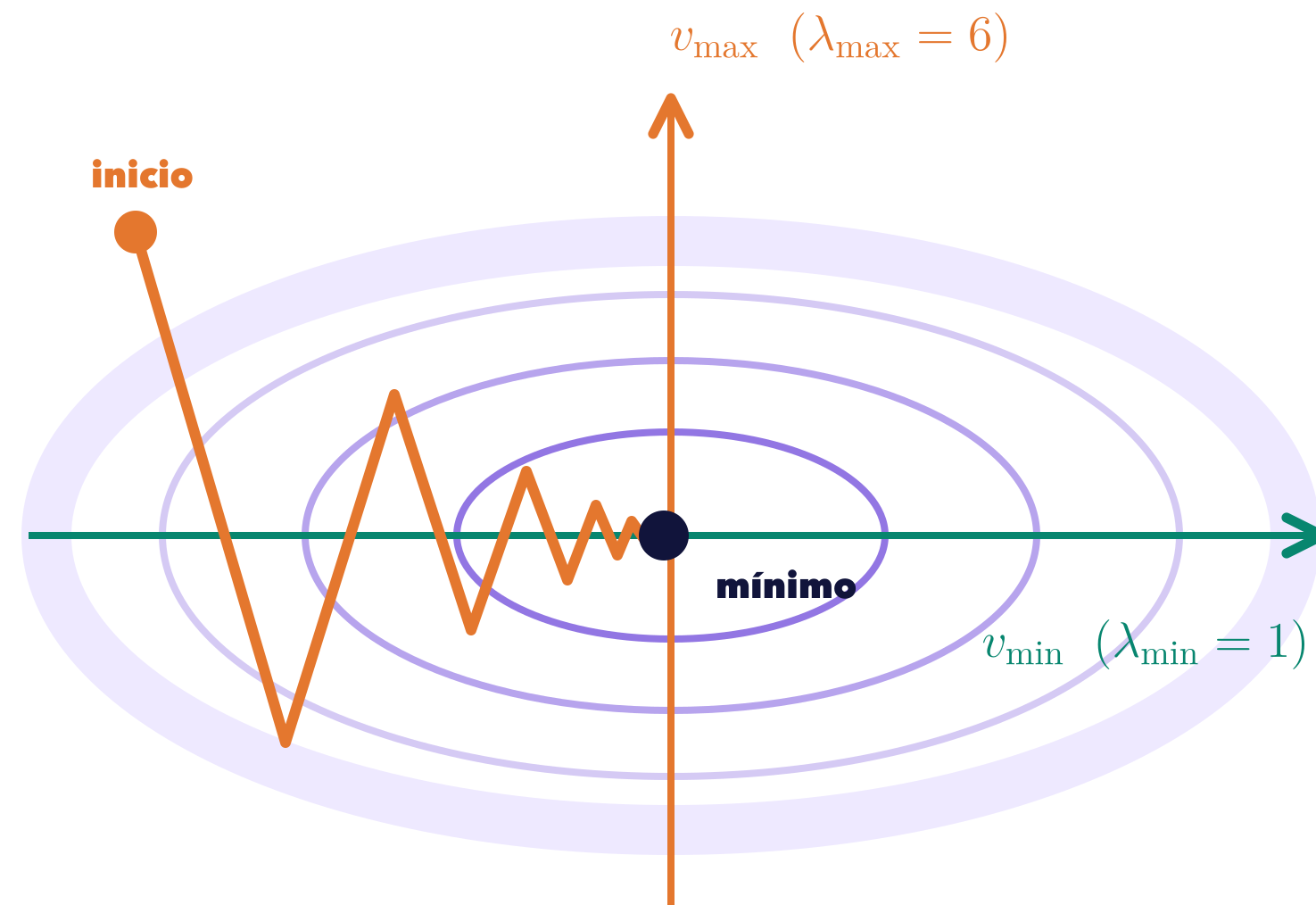
$$|\rho| < 1 \iff 0 < \eta < \frac{2}{a}$$

LA FRONTERA ESTABLE DEPENDE DE LA CURVATURA a

La mayor curvatura limita la tasa estable

Cada dirección propia conserva su factor; una sola debe ser inestable para arruinar el paso.

EL VALLE COMPARTE DOS ESCALAS



LA DIRECCIÓN MÁS CURVA ALTERNA EL SIGNO

LA BASE PROPIA SEPARA LA DINÁMICA

$$L(\theta) = \frac{1}{2}(\theta - \theta^*)^\top H(\theta - \theta^*), \quad H \succ 0$$

$$z_{i,t+1} = (1 - \eta\lambda_i)z_{i,t}$$

Ejemplo visible: $\eta = 0.28$

$$\lambda_{\min} = 1 : \quad 1 - \eta\lambda_{\min} = 0.72$$

$$\lambda_{\max} = 6 : \quad 1 - \eta\lambda_{\max} = -0.68$$

$$0 < \eta < \frac{2}{\lambda_{\max}(H)}$$

$\lambda_{\max}$ **FIJA LA RESTRICCIÓN COMÚN**

ALCANCE: esta cota y la recurrencia son exactas para una cuadrática con $H \succ 0$; el zigzag es una interpretación, no una ley universal.

La tabla de tasas convierte álgebra en predicciones falsables

El código verifica el factor teórico; no decide la clasificación después de observar la traza.

$$a = 4 \implies e_{t+1} = (1 - 4\eta)e_t$$

TASA	FACTOR	PREDICCIÓN ANTES DE EJECUTAR
$\eta = 0.10$	0.60	CONVERGE SIN CAMBIO DE SIGNO
$\eta = 0.25$	0	LLEGA EN UN PASO
$\eta = 0.40$	-0.60	OSCILA Y CONVERGE
$\eta = 0.50$	-1	OSCILA SIN REDUCIR
$\eta = 0.60$	-1.40	DIVERGE

CONTRASTE COMPUTACIONAL

```
def quadratic_gd(theta0, eta, a=4.0):  
    theta = theta0  
    trace = []  
    for _ in range(6):  
        grad = a * theta  
        trace.append(theta)  
        theta -= eta * grad  
    return trace
```

IDENTIDAD EXIGIDA EN CADA PASO

$$e_{t+1} \stackrel{?}{=} (1 - 4\eta)e_t$$

PRIMERO CLASIFICA; DESPUÉS CONTRASTA $e_{t+1} = (1 - 4\eta)e_t$

El mini-batch estima el gradiente completo; no lo copia paso a paso

Una realización puede desviarse aunque el estimador sea correcto en esperanza.

POBLACIÓN COMPLETA

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B \ell_i(\theta)$$

$$\nabla L(\theta) = \frac{1}{B} \sum_{i=1}^B \nabla \ell_i(\theta)$$

contribuciones individuales

$$\{\nabla \ell_1, \nabla \ell_2, \dots, \nabla \ell_B\}$$

full-batch promedia las B contribuciones

UNA REALIZACIÓN DEL ESTIMADOR

MUESTREO UNIFORME SIN REEMPLAZO, DADO EL ESTADO ACTUAL

$$S_t \mid \theta_t \sim \text{Unif}\{S \subset \{1, \dots, B\} : |S| = m\}$$



$$g_t = \frac{1}{m} \sum_{i \in S_t} \nabla \ell_i(\theta_t)$$

la muestra cambia; la definición del estimador no

CASO LÍMITE: TODO EL LOTE

$$S_t = \{1, \dots, B\} \implies g_t = \nabla L(\theta_t)$$

cada mini-batch es una realización, no una copia

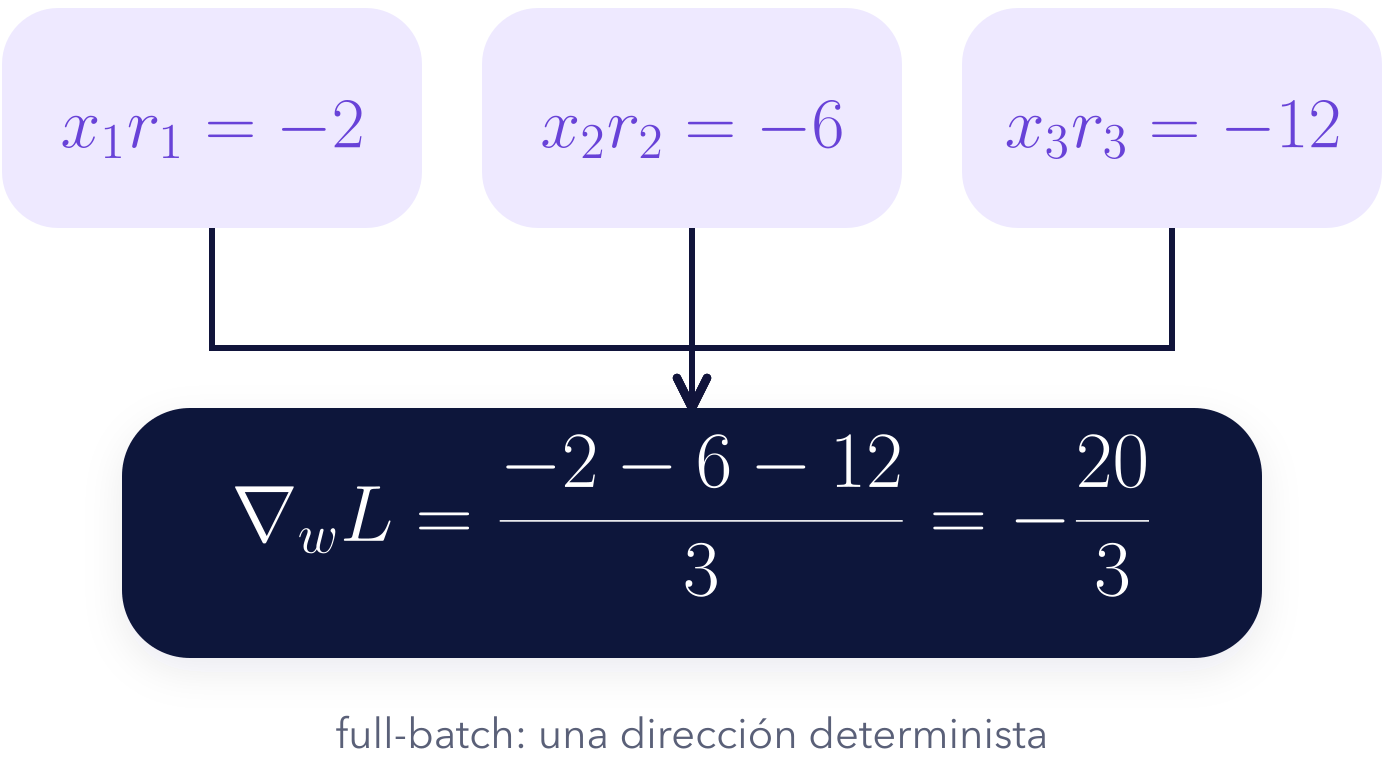
$$\mathbb{E}[g_t \mid \theta_t] = \nabla L(\theta_t)$$

NO SESGADO NO SIGNIFICA $g_t = \nabla L(\theta_t)$ EN CADA PASO

Reducir el lote intercambia costo inmediato por variabilidad

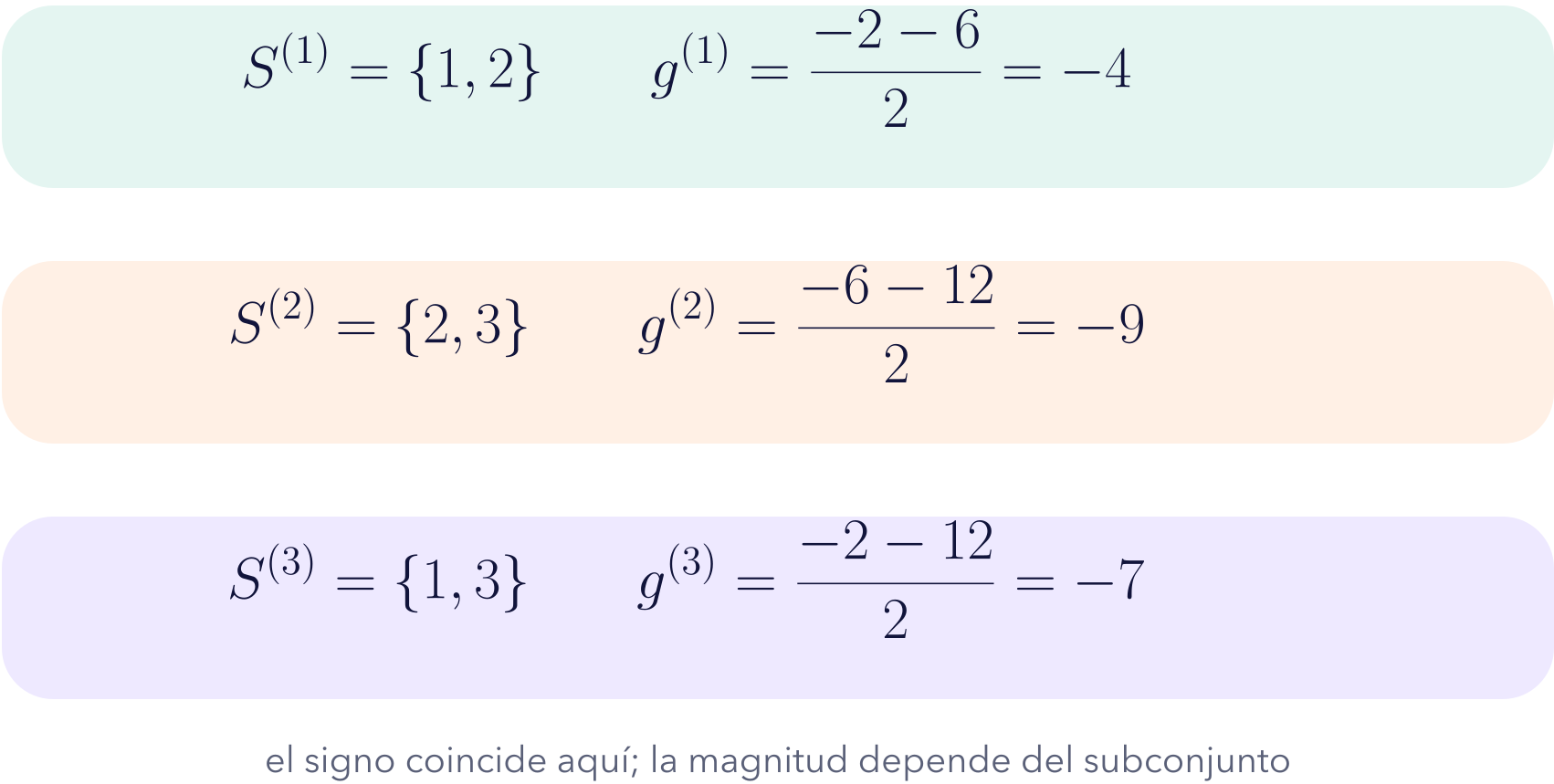
Los índices observados explican por qué dos mini-batches producen magnitudes distintas.

CONTRIBUCIONES DEL LOTE CONDUCTOR



SUBCONJUNTOS REGISTRADOS

$m = 2$



MÁS VARIABILIDAD

MÁS COSTO POR PASO

$m = 1$

$m = 2$

$m = 3$

MENOR m : MENOR COSTO INMEDIATO Y MAYOR VARIABILIDAD; NO EXISTE UN GANADOR UNIVERSAL

Momentum agrega memoria al paso sin cambiar el gradiente

El estado acumulado modifica el paso sin redefinir el gradiente.

GD: SOLO EL GRADIENTE ACTUAL

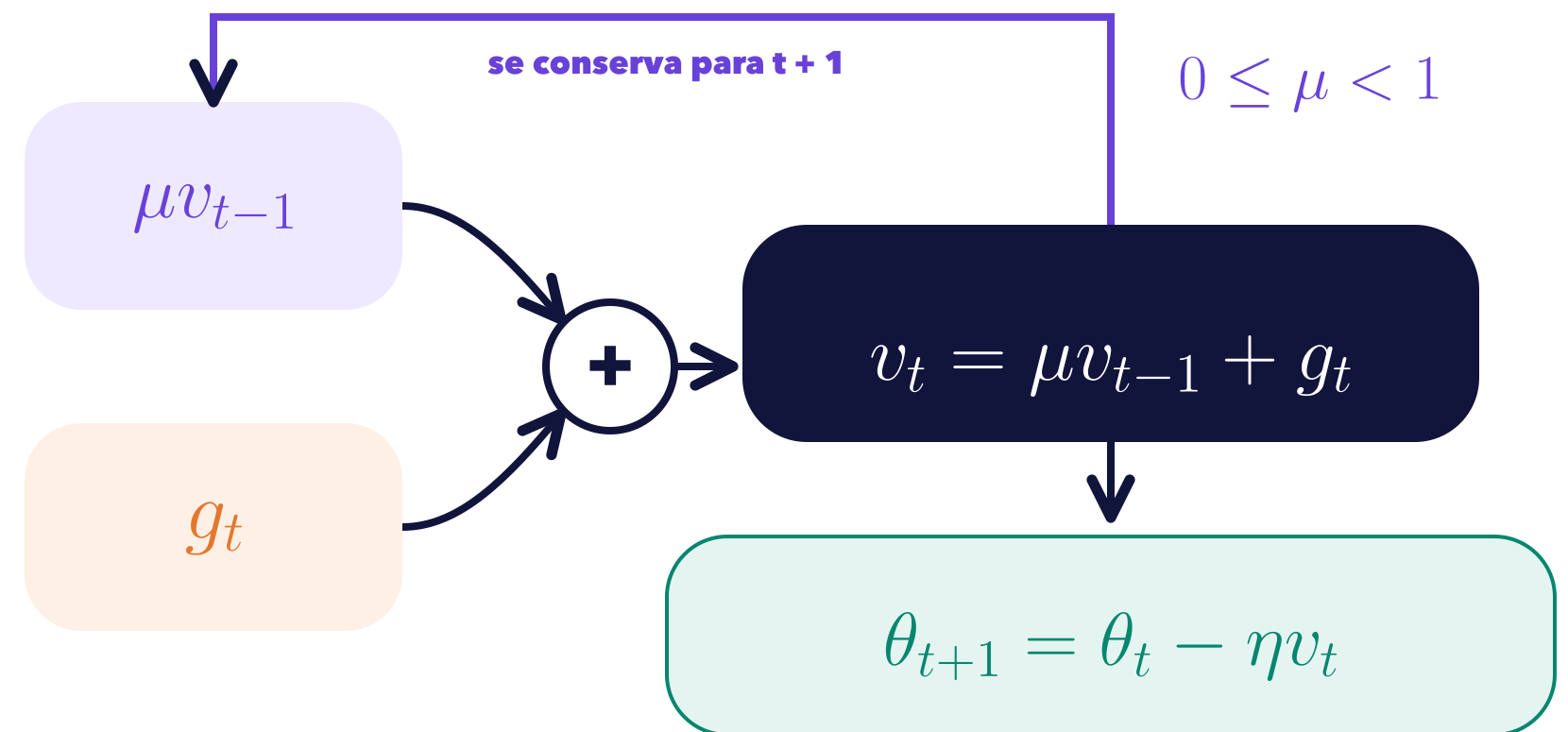
$$\theta_{t+1} = \theta_t - \eta g_t$$



SIN ESTADO PERSISTENTE

el siguiente paso no conserva historia adicional

MOMENTUM: GRADIENTE + MEMORIA



DESENNROLLAR LA RECURRENCIA CONSERVA EL ESTADO INICIAL

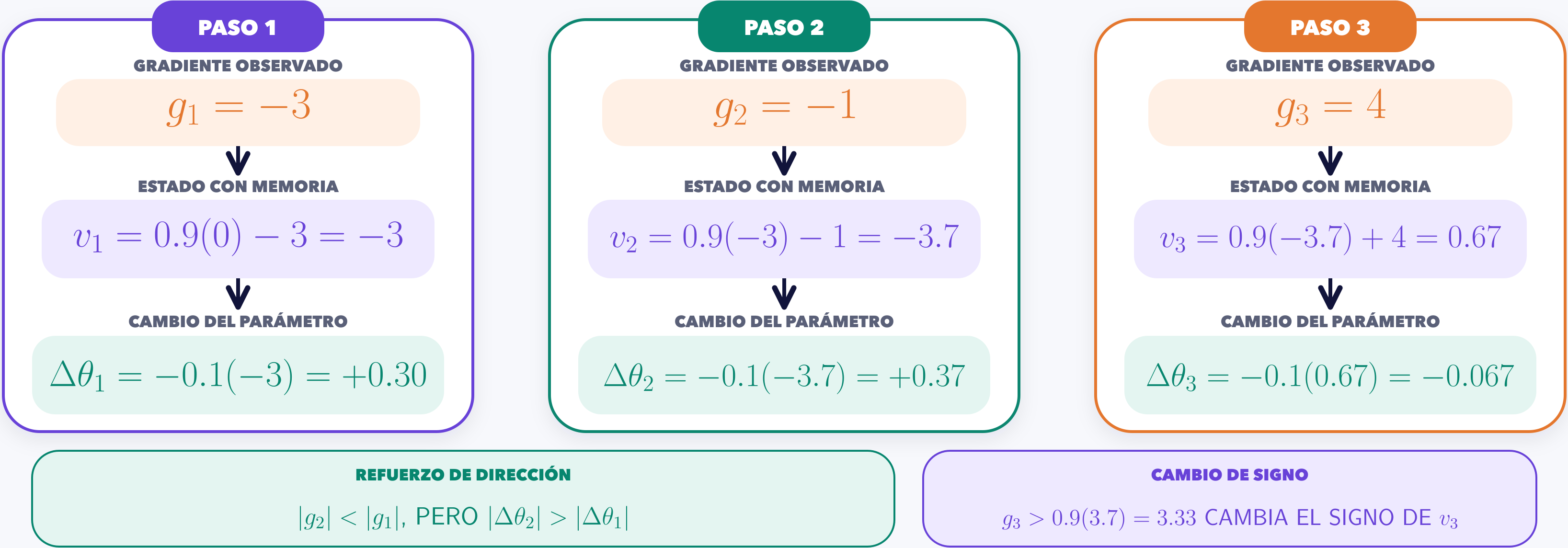
$$v_t = g_t + \underbrace{\mu g_{t-1} + \dots + \mu^{t-1} g_1}_{\text{gradientes observados}} + \underbrace{\mu^t v_0}_{\text{estado inicial}}$$

EL GRADIENTE g_t NO CAMBIA; CAMBIA LA REGLA QUE LO CONVIERTE EN PASO

Momentum puede ampliar un paso aunque el gradiente se reduzca

La memoria conserva dirección: el tamaño del paso no depende solo del gradiente actual.

$v_0 = 0 \quad \mu = 0.9 \quad \eta = 0.1$



EL PASO DEPENDE DE g_t Y DE LA MEMORIA v_{t-1} ; NO SOLO DE $|g_t|$

El momentum_buffer hace visible la memoria del optimizador

El estado observable debe coincidir con la recurrencia; no interpretarse como una caja negra.

EXPERIMENTO CONTROLADO

```
parameter = torch.tensor([0.0],
requires_grad=True)
optimizer = torch.optim.SGD(
[parameter], lr=0.1, momentum=0.9)

for gradient in (-3.0, -1.0, 4.0):
optimizer.zero_grad(set_to_none=True)
parameter.grad = torch.tensor([gradient])
optimizer.step()
state = optimizer.state[parameter][
"momentum_buffer"].detach().clone()
print(parameter.item(), state.item())
```

GRADIENTES INYECTADOS: VERIFICACIÓN, NO ENTRENAMIENTO

$$b_t = 0.9b_{t-1} + g_t$$

PASO	GRADIENTE	BUFFER	PARÁMETRO
1	-3	-3	0.30
2	-1	-3.7	0.67
3	4	0.67	0.603

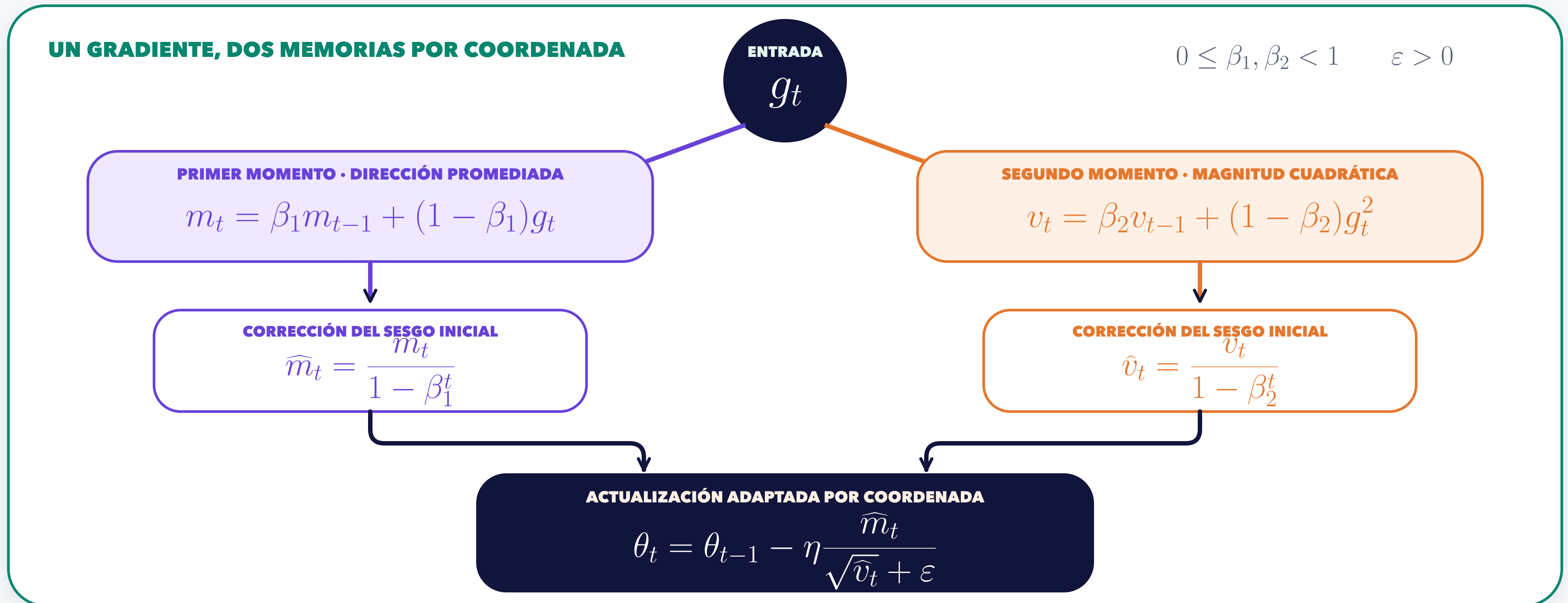
CONTRASTE DECISIVO

EN $t = 2$: $g_2 = -1$, PERO $buffer_2 = -3.7$

EL BUFFER CERTIFICA MEMORIA; NO DEBE LEERSE COMO UNA COPIA DE g_t

Adam transforma el mismo gradiente con dos estados corregidos

El primer momento sigue dirección; el segundo sigue magnitud cuadrática.



AUTOGRAD ENTREGA g_t ; ADAM USA ESTADO PARA TRANSFORMARLO EN UN PASO

**NO IMPLICA
SUPERIORIDAD UNIVERSAL**

Autograd calcula g; Adam conserva estado y decide el paso

El gradiente existe antes de step(); el estado aparece cuando el optimizador actualiza.

SECUENCIA MÍNIMA

```
parameter = torch.tensor([1.0],  
requires_grad=True)  
optimizer = torch.optim.Adam(  
[parameter], lr=0.01)  
  
optimizer.zero_grad(set_to_none=True)  
loss = 0.5 * (parameter - 3.0).pow(2).sum()  
loss.backward()  
gradient_before_step = parameter.grad.clone()  
optimizer.step()  
state_after_step = optimizer.state[parameter]
```

OBSERVAR ANTES Y DESPUÉS DE `optimizer.step()`

AUTOGRAD • DEFINE EL GRADIENTE

`loss.backward()`

$$p_0 = 1, y = 3, L = \frac{1}{2}(p - y)^2 = 2, g = -2$$

ANTES DE `step()`: `parameter.grad = -2`

`optimizer.step()`

CAMBIO DEL PARÁMETRO

$$p_1 \approx 1.01 \quad \Delta p \approx +0.01$$

ESTADO DE ADAM

contador + dos momentos
`step = 1`

`exp_avg = -0.2`

`exp_avg_sq = 0.004`

`parameter.grad = -2` DESPUÉS DE `step()`

MISMA PÉRDIDA Y PARÁMETROS $\Rightarrow$ AUTOGRAD FIJA g ; ADAM SOLO TRANSFORMA EL PASO

train/eval y grad/no-grad son interruptores ortogonales

El modo responde cómo actúa el módulo; autograd responde si registra operaciones.

GRADIENTES HABILITADOS

CONTEXTO no_grad

model.train()

comportamiento
de entrenamiento

TRAIN + GRAD

módulos sensibles usan su rama de entrenamiento

training = True requires_grad = True
grad_fn ≠ None

TRAIN + NO-GRAD

modo train sin historial de autograd

training = True requires_grad = False
grad_fn = None

train() with torch.no_grad():

model.eval()

comportamiento
de evaluación

EVAL + GRAD

modo eval con historial de autograd

training = False requires_grad = True
grad_fn ≠ None CON LINEAR

model.eval() y=model(x)

EVAL + NO-GRAD

evaluación sin historial

training = False requires_grad = False
grad_fn = None

train/eval CONTROLA COMPORTAMIENTO DEL MÓDULO

grad/no_grad CONTROLA REGISTRO DEL GRAFO

Dropout consulta training; una capa lineal no lo hace

La misma entrada revela un cambio de comportamiento, no un cambio de autograd.

ENTRADA COMÚN · SEMILLA 8

$$X_{\text{mode}} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}^T$$

TRAIN · DOS MÁSCARAS REPRODUCIBLES

`dropout.train()` `training = True`

EJECUCIÓN 1

$$\widetilde{X}_1 = [0, 0, 2, 2, 0, 2]^T$$

EJECUCIÓN 2

$$\widetilde{X}_2 = [0, 2, 2, 0, 2, 0]^T$$

$$p = 0.5 \quad \Rightarrow \quad \tilde{x}_i \in \{0, 2\}$$

LAS MÁSCARAS PUEDEN DIFERIR

EVAL · IDENTIDAD EXACTA

`dropout.eval()` `training = False`

sin máscara aleatoria · sin reescalamiento

$$X_{\text{eval}} = [1, 1, 1, 1, 1, 1]^T = X_{\text{mode}}$$

`requires_grad = True` SE CONSERVA

EL MODO NO APAGA AUTOGRAD

CONTROL NEGATIVO

`nn.Linear` APLICA LA MISMA TRANSFORMACIÓN EN `train` Y `eval`

El reescalamiento de Dropout preserva la expectativa

Con $p=0.5$, una entrada unitaria vale 0 o 2 en train, pero siempre 1 en eval.

REGLA DURANTE ENTRENAMIENTO

$$\tilde{x} = \frac{m}{1-p}x, \quad m \sim \text{Bernoulli}(1-p)$$

$x = 1 \quad p = 0.5 \quad \frac{1}{1-p} = 2$

LA UNIDAD SE ELIMINA

$$m = 0 \Rightarrow \tilde{x} = 0$$
$$\Pr(m = 0) = 0.5$$

LA UNIDAD SOBREVIVE Y SE ESCALA

$$m = 1 \Rightarrow \tilde{x} = 2$$
$$\Pr(m = 1) = 0.5$$

EVAL

identidad exacta

$$\tilde{x}_{\text{eval}} = x = 1$$

$$\Pr(\tilde{x}_{\text{eval}} = 1) = 1$$



SIN AZAR

BALANCE EN PROMEDIO

$$\mathbb{E}[\tilde{x}] = 0(0.5) + 2(0.5) = 1 = x$$



$\mathbb{E}[\tilde{x}] = x$ NO SIGNIFICA $\tilde{x} = x$ EN CADA MUESTRA

eval() no apaga autograd: hay que inspeccionar evidencia

Valores, estado del módulo y seguimiento del grafo son observaciones distintas.

TRAZA REPRODUCIBLE

```
torch.manual_seed(8)

dropout = nn.Dropout(p=0.5)
X_mode = torch.ones(6, 1,
requires_grad=True)

dropout.train()
train_output_1 = dropout(X_mode)
train_output_2 = dropout(X_mode)

dropout.eval()
eval_output = dropout(X_mode)
eval_output.sum().backward()
```

PYTORCH 2.13.0 • SEMILLA 8

AUDITORÍA CAUSAL

1 • MODO Y TRANSFORMACIÓN

dropout.training = False
eval_output = X_{mode}

2 • SEGUIMIENTO DE AUTOGRAD

eval_output.requires_grad = True
grad_fn = None AQUÍ: LA IDENTIDAD DEVUELVE LA HOJA

3 • BACKWARD FUNCIONA

eval_output.sum().backward() $\Rightarrow X_{\text{mode}}.\text{grad} = \mathbf{1}_6$

4 • UNA OPERACIÓN REAL CREA grad_fn

Dropout $\rightarrow$ Linear EN EVAL $\Rightarrow$ grad_fn = AddmmBackward0

**NO
CONFUNDIR**

eval()
 $\neq$
no_grad()

son dos
interruptores

eval() CAMBIA EL MÓDULO; torch.no_grad() CAMBIA EL REGISTRO

no_grad corta el historial, no cambia la entrada

La propiedad de la hoja no basta: importa dónde se registró cada operación.

EXPERIMENTO CONTROLADO

```
x = torch.tensor([2.0],  
requires_grad=True)
```

```
y = 3.0 * x
```

```
with torch.no_grad():  
    z = 3.0 * x
```

```
q = 4.0 * z
```

MISMA HOJA • DOS HISTORIAS



no_grad CORTA EL HISTORIAL EN z : OPERAR DESPUÉS NO RECONECTA q CON x

Modo y registro producen cuatro combinaciones distintas

Una capa lineal controla el experimento: los valores no deben distraer de las propiedades.

CUATRO EJECUCIONES CONTROLADAS

```
model = nn.Linear(1, 1)
x = torch.ones(2, 1)
model.train()
out_train_grad = model(x)
model.eval()
out_eval_grad = model(x)
model.train()
with torch.no_grad():
    out_train_no_grad = model(x)
model.eval()
with torch.no_grad():
    out_eval_no_grad = model(x)
```

MISMA x • MISMA CAPA • DOS INTERRUPTORES

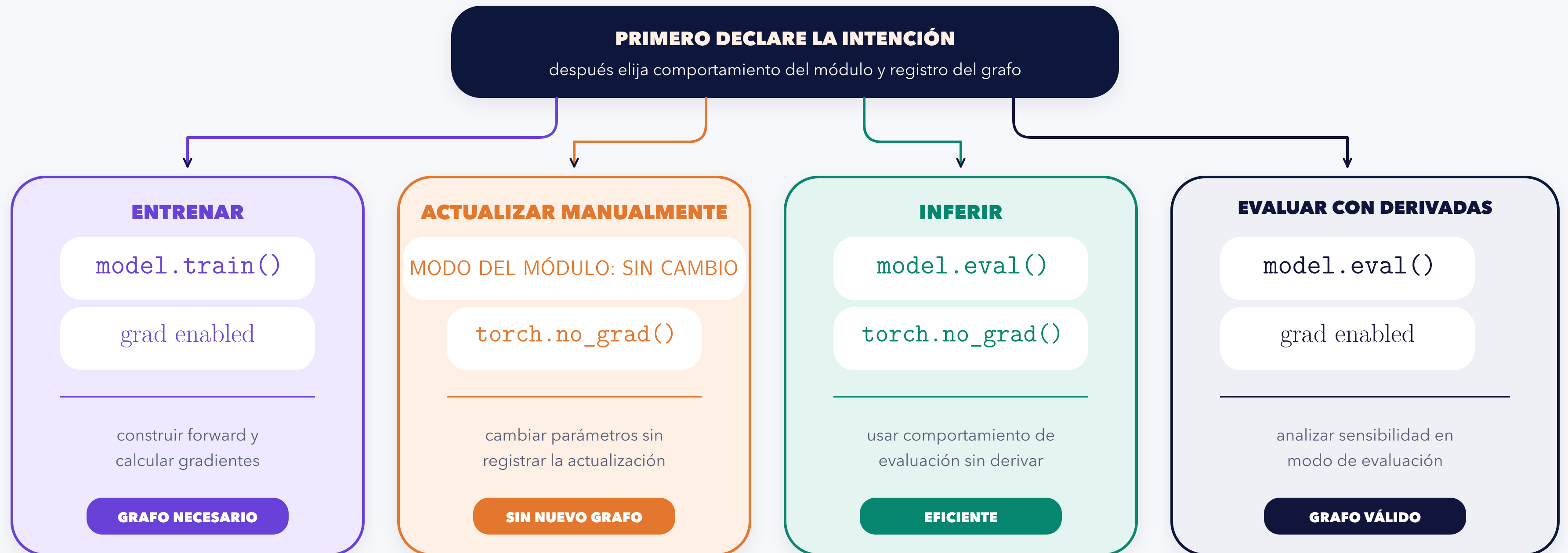
RESULTADO OBSERVABLE

TRAIN + GRAFO out_train_grad	model.training = True	requires_grad = True
EVAL + GRAFO out_eval_grad	model.training = False	requires_grad = True
TRAIN + SIN GRAFO out_train_no_grad	model.training = True	requires_grad = False
EVAL + SIN GRAFO out_eval_no_grad	model.training = False	requires_grad = False

nn.Linear CONSERVA LA TRANSFORMACIÓN: LA EVIDENCIA ESTÁ EN training Y requires_grad

La intención decide el modo y el contexto de gradientes

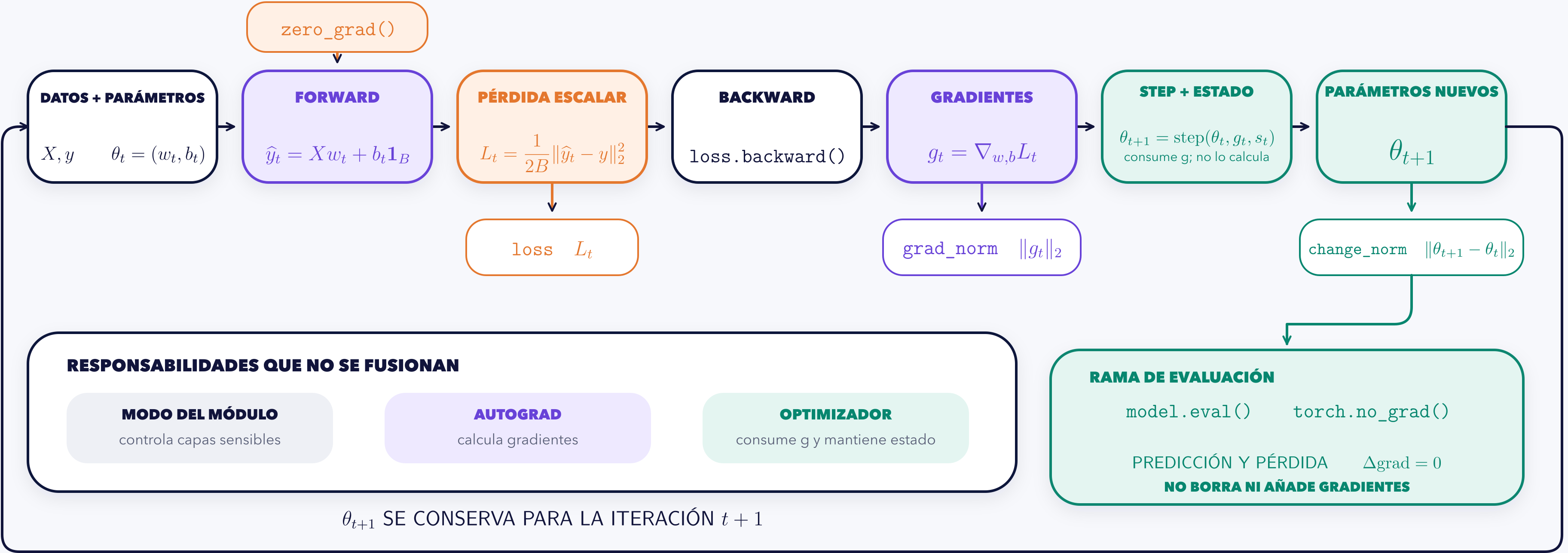
Entrenar, actualizar e inferir no son sinónimos operativos.



`.data` NO EXPRESA UNA INTENCIÓN: ELIJA MODO Y CONTEXTO DE FORMA EXPLÍCITA

El ciclo mínimo separa responsabilidades y deja evidencia

Forward, backward y step forman una cadena causal; no son una sola operación.



LOSS • VALOR DEL OBJETIVO

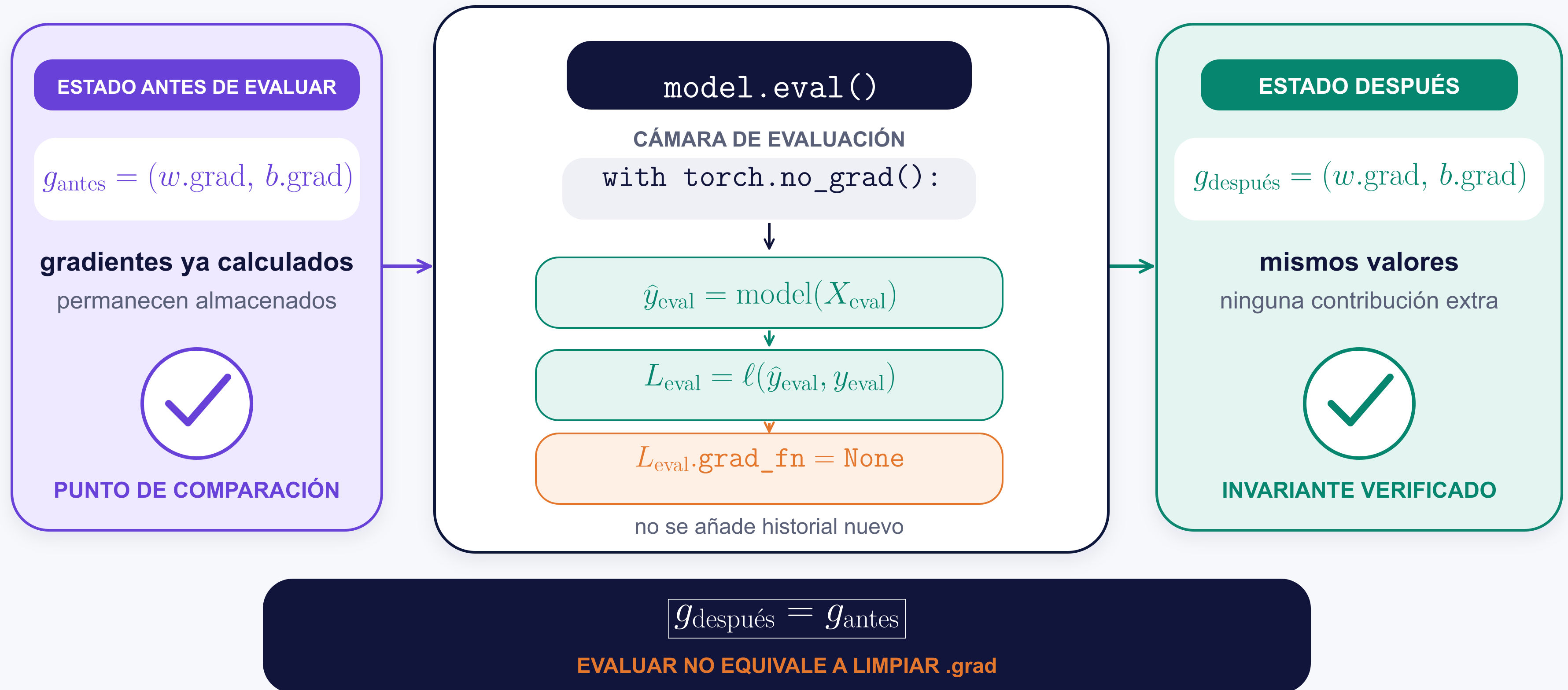
GRAD_NORM • SENSIBILIDAD

CHANGE_NORM • CAMBIO REAL

Una curva de pérdida sola no identifica dónde falló el ciclo.

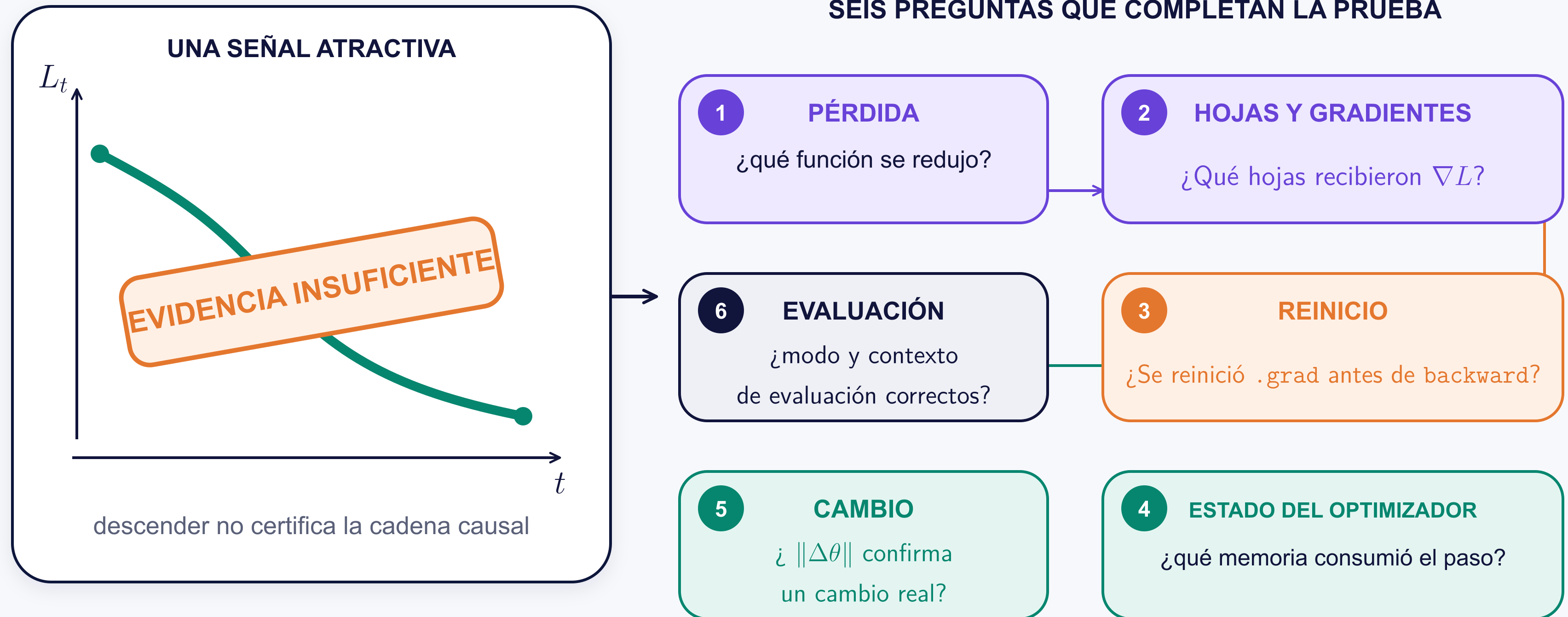
Evaluar no agrega gradientes ni borra los previos

El contexto evita historial nuevo; los gradientes existentes deben comprobarse antes y después.



Una curva descendente sola no prueba un protocolo correcto

La forma de la curva es una señal; la cadena causal completa es la evidencia.



La prueba válida conecta pérdida → grafo → reinicio → estado → cambio → evaluación.

Broadcasting silencioso puede cambiar la función objetivo

Una resta que ejecuta sin error puede expandir B residuales hasta una matriz $B \times B$.

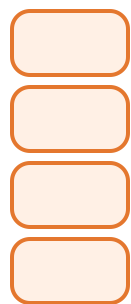
SIN ERROR DE EJECUCIÓN · CON ERROR DE OBJETIVO

prediction: $(B,)$



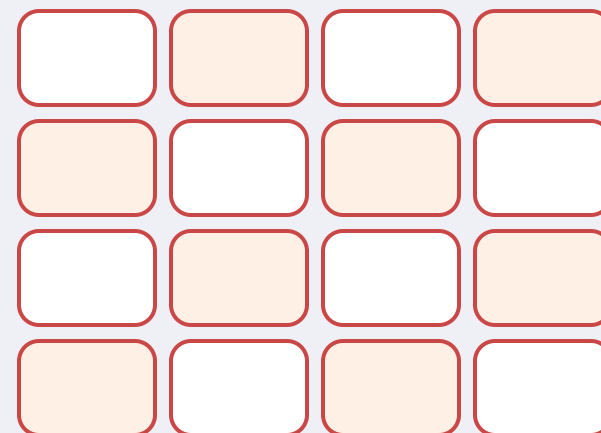
vector fila

target y : $(B, 1)$



vector columna

$$(\hat{y}_j - y_i)_{i,j=1}^B \in \mathbb{R}^{B \times B}$$



$B \times B$ residuales, no B

$$L = \text{mean}_{i,j} (\hat{y}_j - y_i)^2$$

otra función objetivo

FORMA ALINEADA

prediction: $(B, 1)$



target y : $(B, 1)$



$\hat{y} - y : (B, 1)$

un residual por ejemplo

la reducción conserva intención

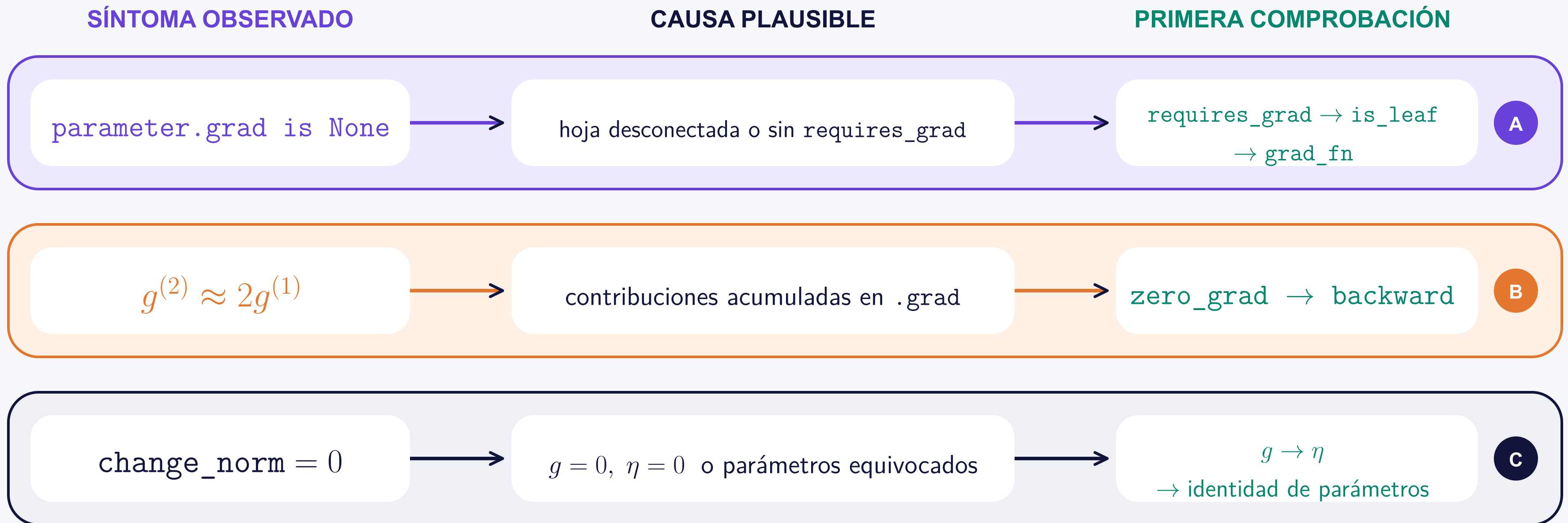


```
assert prediction.shape == y.shape
```

PRIMER FRENO DIAGNÓSTICO: COMPARAR FORMAS ANTES DE LA PÉRDIDA

Tres síntomas exigen tres rutas de diagnóstico

La primera comprobación cambia con el síntoma; una corrección refleja puede ocultar la causa.



“Bajar η ” no es una corrección común: primero identifique la causa.

SÍNTOMA ≠ CAUSA ≠ CORRECCIÓN

El Hackathon exige evidencia causal, no solo código que corre

Una solución defendible asciende por seis evidencias; ejecutar es solo una de ellas.

